158.337 Database Development

Week 4 - Normalization of Database Tables

Chapter 6

Table structures are poor whenever they promote uncontrolled data redundancy.
From an information management point of view, possibly the most vexing and destructive problems are created through uncontrolled data redundancies. Such redundancies produce update and delete anomalies that create data integrity problems. The loss of data integrity can destroy the usefulness of the data within the database.

Normalization does not eliminate data redundancy; instead, it produces the carefully *controlled* redundancy that lets us properly link database tables. Without normalization principles and procedures, we lack evaluation standards and must rely on experience (and yes, some intuition) to minimize the probability of generating data integrity problems. The problem with relying on experience is that we usually learn from experience by making errors. While we're learning, who and what will be hurt by the errors we make?

Relying on intuition may work reasonably well for some, but intuitive work habits seldom create design consistency. Worse, you can't teach intuition to those who follow in your database footsteps.

In short, the normalization principles and rules drastically decrease the likelihood of producing bad table structures, they help standardize the process of producing good tables, and they make it possible to transmit skills to the next generation of database designers.

The normalization process helps in identifying potential problems (reporting anomalies and data integrity issues) in the design process. To correct anomalies in a database, you must convert tables to various types of normal forms. The most normal forms are first normal form (1NF), second normal form (2NF), third normal form (3NF), Boyce-Codd normal form (BCNF) and fourth normal form (4NF). Normalization is a progression in which a table that is in 1NF is better than a table that is not in 1NF, a table in 2NF is better than a table in 1NF, and so on….

As you normalize tables, you will have more clarity on existing dependencies (functional, partial and transitive) in your table structures. The normalization process takes you thru the following steps in defining table structures:

1 NF

A table is in 1NF when all the key attributes are defined (no repeating groups in the table) and when all remaining attributes are dependent on the primary key. However, a table in 1NF still may contain partial dependencies, i.e., dependencies based on only part of the primary key and/or transitive dependencies that are based on a non-key attribute.

2NF

A table is in 2NF when it is in 1NF and it includes no partial dependencies. However, a table in 2NF may still have transitive dependencies, i.e., dependencies based on attributes that are not part of the primary key.

3NF
A table is in 3NF when it is in 2NF and it contains no transitive dependencies.

BCNF
A table is in Boyce-Codd Normal Form (BCNF) when it is in 3NF and every determinant in the table is a candidate key. For example, if the table is in 3NF *and it contains a nonprime attribute that determines a prime attribute*, the BCNF requirements are not met.

4NF
Table is in fourth normal form (4NF) when both of the following are true (1) It is in 3NF and (2) No multiple sets of multivalued dependencies

The textbook chapter contains some very good examples on Normalization. Through normalization, we increase the number of entities (tables), and … yes… the design also gets complicated. Normalization is likely to suggest the existence of entities that you may not have been considered during the modeling process.

So, your next tutorial exercise is about

- Understanding table structures
- Evaluate dependency diagrams
- Give a set of data – You have to make it normal by evaluating and correcting table structures.
- Create the tables in your Oracle account